

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

**Redovne studije
Smjer : “Inženjering Informacionih Tehnologija“**

**Predmet:
Softverski inženjering**

PROCES RAZVOJA SOFTVERA

(Seminarski rad)

**Predmetni nastavnik
Doc. dr Dalibor Drljača**

**Student: Nikolina Bekić
Br. Indeksa: 21-22-FIT-IIT1-PRO-240**

Banja Luka, jun 2026

SADRŽAJ

UVOD	1
1. POJAM PROCESA RAZVOJA SOFTVERA	2
1.1. Proces razvoja i životni ciklus	2
1.2. Osnovne faze razvoja softvera	3
1.3. Kvalitet i značaj procesa	3
2. KLASIČNI (PLANOM VOĐENI) MODELI PROCESA	5
2.1. Vodopadni model i njegove varijante	5
2.2. V-model i naglasak na verifikaciji i validaciji	6
2.3. Iterativno-inkrementalni i spiralni pristup	7
3. AGILNI PROCESI I SAVREMENI NAČINI RADA	9
3.1. Agilne vrijednosti i principi	9
3.2. Scrum kao okvir za razvoj proizvoda	9
3.3. DevOps, CI/CD i uticaj na proces razvoja	10
4. UPRAVLJANJE PROCESOM I UNAPREĐENJE	12
4.1. Upravljanje zahtjevima i promjenama	12
4.2. Konfiguraciono upravljanje i praćenje rada	12
4.3. Unapređenje procesa i zrelost	13
ZAKLJUČAK	14
POPIS SLIKA	15
CITATNI IZVORI	16

UVOD

Proces razvoja softvera je organizovan skup aktivnosti kojim se od početne ideje dolazi do gotovog i funkcionalnog softverskog sistema. On obuhvata prikupljanje i analizu zahtjeva, projektovanje sistema, implementaciju, testiranje, isporuku i kasnije održavanje.

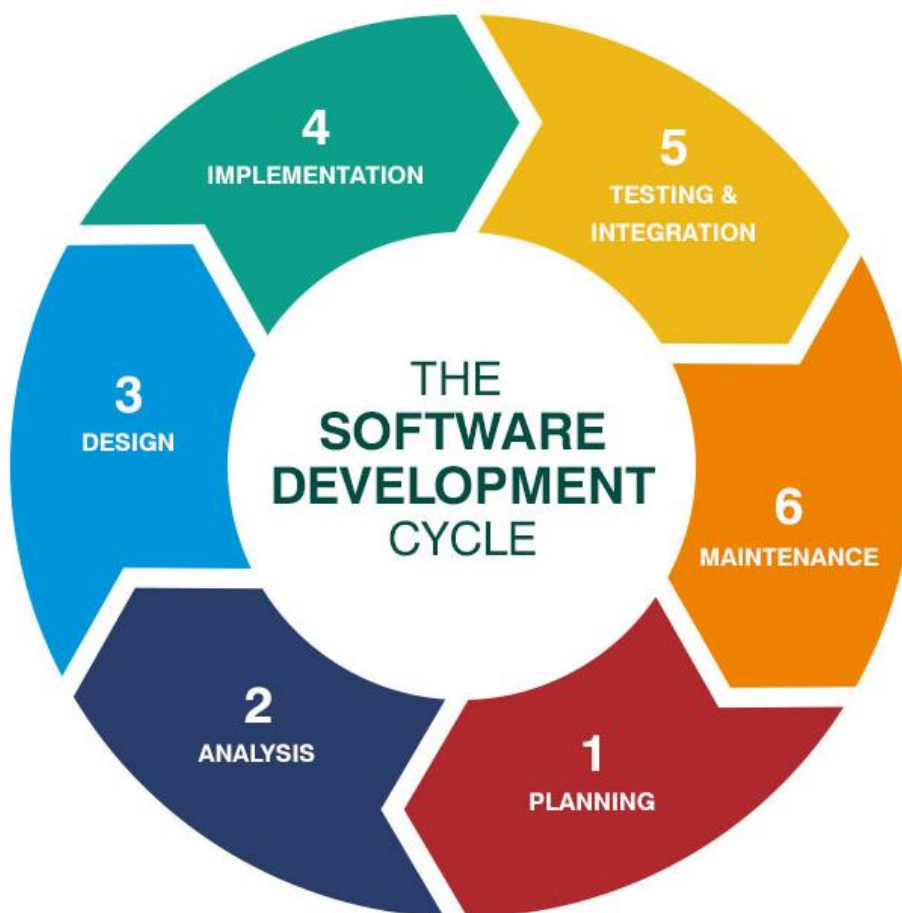
U osnovi, proces razvoja softvera definiše način na koji tim radi i sarađuje tokom cijelog životnog ciklusa sistema. Njegova uloga je da se rad ne odvija nasumično, nego po jasno definisanim koracima, kako bi se smanjile greške i povećala kontrola nad promjenama. Razvoj softvera se ne svodi samo na pisanje koda. Kod je samo jedan dio procesa, dok su jednako važni i planiranje, komunikacija u timu, provjera kvaliteta i priprema za rad u produkciji. Svaka faza ima svoju svrhu i utiče na krajnji rezultat.

Postojanje definisanog procesa omogućava da se zahtjevi bolje razumiju i dokumentuju, da se promjene prate i odobravaju, te da se greške otkrivaju ranije u razvoju. Na taj način se smanjuje rizik od problema u kasnijim fazama, posebno nakon isporuke sistema korisnicima. U praksi, procesi razvoja mogu biti različiti, od klasičnih do agilnih pristupa, ali cilj je uvijek isti, a to je isporuka stabilnog i upotrebljivog softvera koji odgovara potrebama korisnika.

1. POJAM PROCESA RAZVOJA SOFTVERA

1.1. Proces razvoja i životni ciklus

Proces razvoja softvera predstavlja skup aktivnosti i pravila koji određuju način izrade softvera. On definiše kako se prikupljaju zahtjevi, kako se softver projektuje, kako se implementira, testira i na kraju isporučuje i održava. U praksi, proces razvoja služi da se rad organizuje i da se smanji mogućnost grešaka. Bez definisanog procesa, razvoj se svodi na pojedinačne odluke programera, što često dovodi do neusklađenosti i problema u kasnijim fazama.



Slika 1 - Dijagram faza životnog ciklusa razvoja softvera (SDLC)

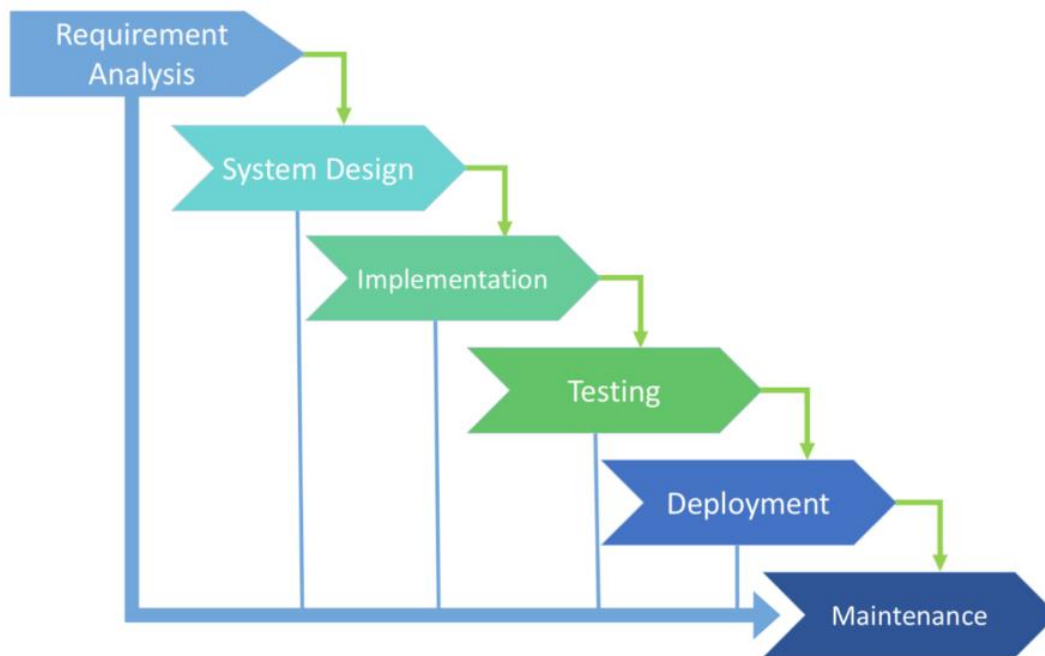
Povezan pojam je životni ciklus softvera. On opisuje sve faze kroz koje softver prolazi od početne ideje do povlačenja iz upotrebe. Te faze su obično analiza, dizajn, implementacija, testiranje, isporuka, korištenje i održavanje. Proces razvoja je dio tog životnog ciklusa. On se najviše odnosi na faze izrade i pripreme softvera, ali ima uticaj i na kasnije faze. Ako je razvoj loše organizovan, održavanje i nadogradnja postaju teži i skuplji.

U razvoju softvera učestvuje više uloga. Najčešće su to analitičari, programeri, testeri i osobe zadužene za isporuku i održavanje sistema. U većim timovima postoje i projekt menadžeri ili product owneri koji definišu prioritete i prate napredak. Važno je shvatiti da proces ne ograničava

rad, nego ga organizuje. On daje osnovni okvir u kojem tim zna šta treba da uradi i kojim redoslijedom. Na taj način se smanjuje haos i povećava stabilnost projekta.

1.2. Osnovne faze razvoja softvera

Razvoj softvera se obično dijeli na nekoliko osnovnih faza. Prva faza je prikupljanje i analiza zahtjeva. U ovoj fazi se definiše šta sistem treba da radi. Loše definisani zahtjevi su jedan od najčešćih uzroka problema u kasnijem razvoju. Druga faza je dizajn sistema. U dizajnu se određuje struktura softvera, način rada modula i komunikacija između komponenti. Ova faza služi da se napravi plan prije pisanja koda. Treća faza je implementacija. To je faza u kojoj se piše programski kod. U ovoj fazi su važni standardi pisanja koda, verzionisanje i kontrola izmjena. Bez toga dolazi do neusklađenosti i teškoća u kasnijem održavanju. Četvrta faza je testiranje. Testiranjem se provjerava da li softver radi kako je planirano. Testiranje se ne radi samo na kraju, nego i tokom razvoja kroz različite nivoe provjera. Peta faza je isporuka softvera. U ovoj fazi se softver priprema i pušta u produkcijsko okruženje. Isporuka mora biti kontrolisana jer greške u ovoj fazi mogu direktno uticati na korisnike.

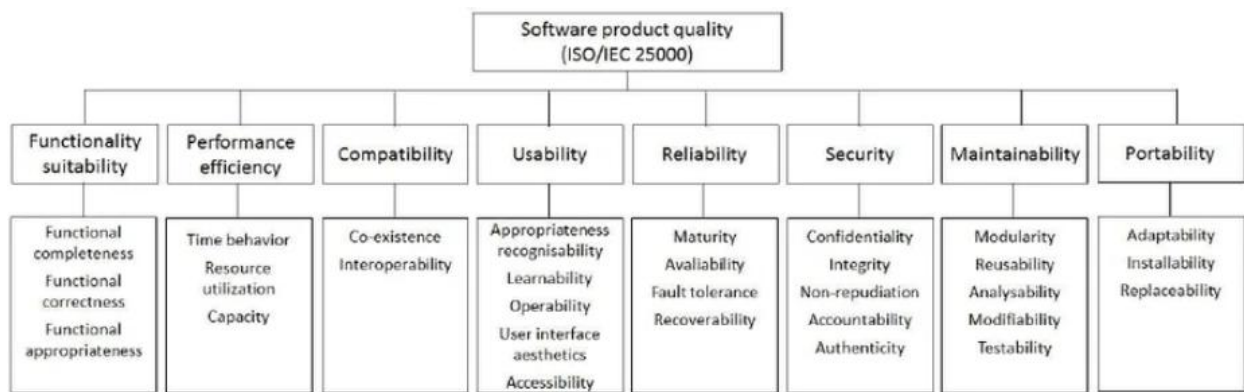


Slika 2 - Jednostavan dijagram faza razvoja softvera

Nakon isporuke dolazi održavanje. Održavanje uključuje ispravljanje grešaka, prilagođavanje sistema i dodavanje novih funkcionalnosti.

1.3. Kvalitet i značaj procesa

Kvalitet softvera se odnosi na to koliko sistem ispunjava očekivanja korisnika i tehničke zahtjeve. Kvalitet se ne odnosi samo na to da li program radi, nego i na stabilnost, pouzdanost, sigurnost i lakoću održavanja. Proces razvoja ima veliki uticaj na kvalitet. Ako je proces dobro definisan, greške se otkrivaju ranije, promjene se uvode kontrolisano, a sistem je stabilniji. Ako proces ne postoji ili nije jasan, kvalitet zavisi od pojedinaca i njihovog iskustva.



Slika 3 - Dijagram ISO 25010 modela kvaliteta softvera

Dobar proces omogućava i bolju organizaciju rada. Tim zna šta se radi, u kojoj fazi se projekat nalazi i šta slijedi dalje. To smanjuje nesporazume i poboljšava komunikaciju unutar tima. Proces mora biti prilagođen projektu. Previše strog proces može usporiti rad, dok premalo strukture vodi u haos. Zbog toga se proces bira prema veličini tima, složenosti sistema i vrsti projekta.

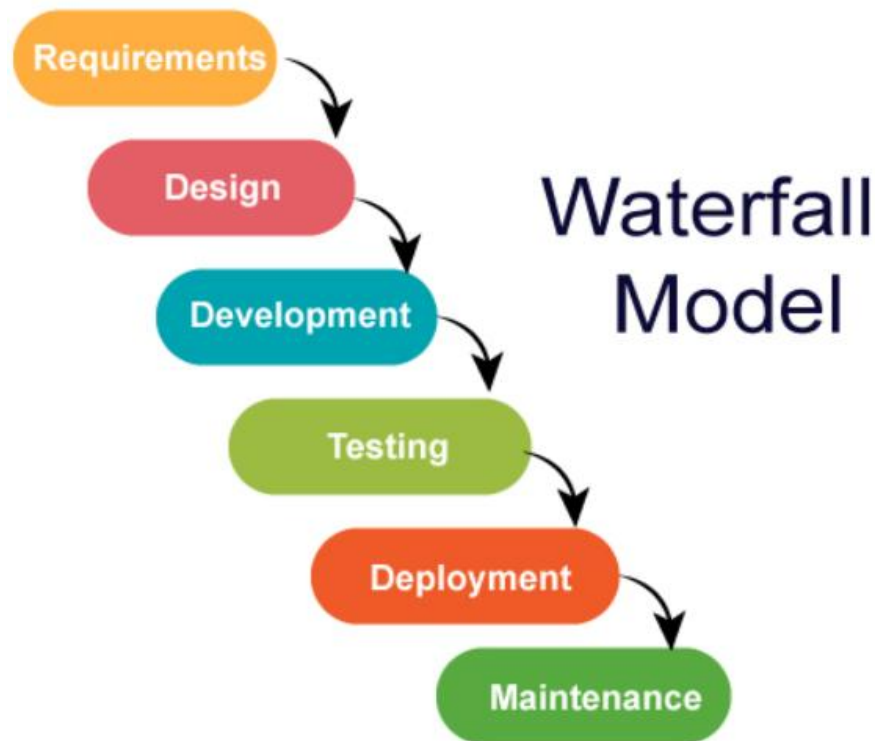


Slika 4 - Dijagram PDCA ciklusa za kvalitet softvera (Plan–Do–Check–Act)

2. KLASIČNI (PLANOM VOĐENI) MODELI PROCESA

2.1. Vodopadni model i njegove varijante

“Vodopadni model je jedan od najpoznatijih klasičnih modela procesa. Ideja je da se radi kroz faze koje slijede jedna drugu: zahtjevi, dizajn, implementacija, testiranje, isporuka. Prednost ovog pristupa je preglednost i jasna dokumentacija, što može biti korisno kada su zahtjevi stabilni i kada se radi u okruženju gdje se traži formalnost” [1].



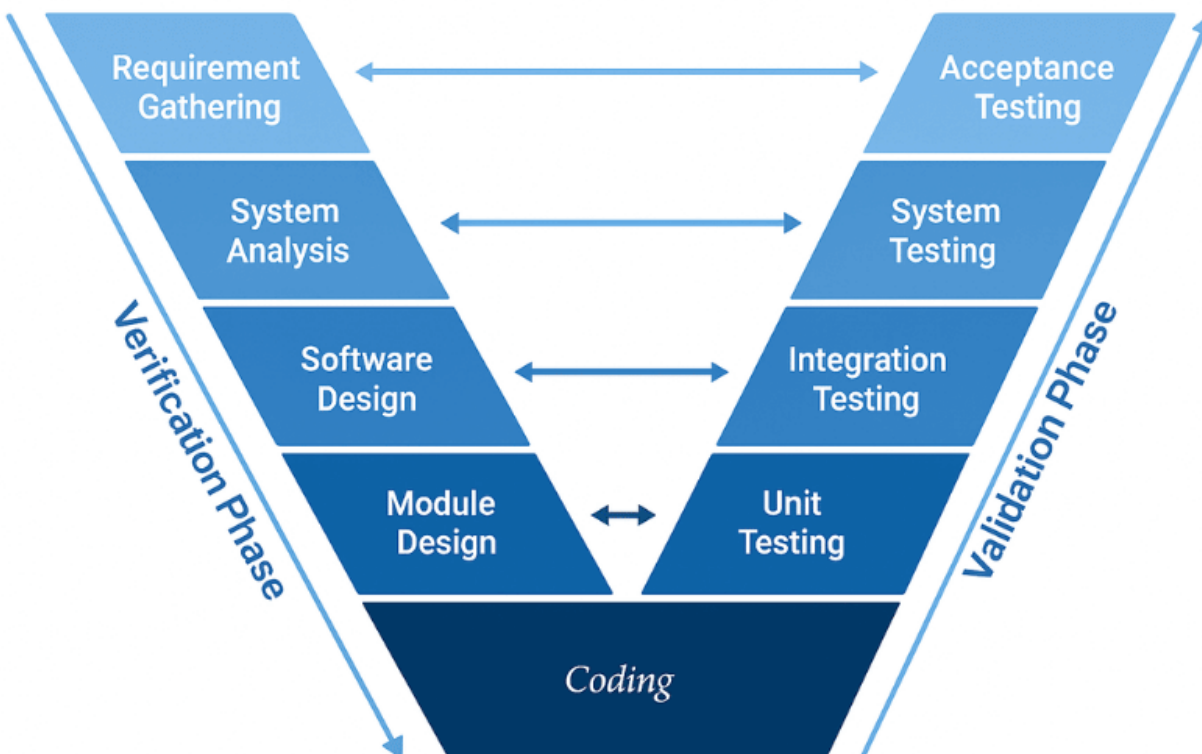
Slika 5 - Dijagram vodopadnog modela (Waterfall model)

Međutim, u praksi se vodopad često kritikuje jer pretpostavlja da se zahtjevi mogu potpuno razumjeti na početku. U stvarnom životu zahtjevi se mijenjaju: korisnik shvati da mu treba nešto drugo, pojavi se konkurencija, promijeni se zakon, ili se tek kroz prototip vidi da je prvobitna ideja bila loša. “Ako model ne predviđa povratne petlje, promjene postaju skupe, jer se vraćamo u ranu fazu nakon što je već mnogo urađeno” [1].

“Zbog toga su nastale razne varijante vodopada koje uvode povratne korake i parcijalne iteracije. Čak i kada se radi planom vođeno, timovi često rade “mini iteracije” unutar faza, npr. prvo detaljno obrade dio zahtjeva, pa dizajniraju i implementiraju taj dio, umjesto da čekaju da sve bude završeno odjednom. To pokazuje da je u praksi rijetko koji proces potpuno “čist” model, nego je mješavina prilagođena projektu” [2]. Vodopad je i dalje prisutan u oblastima gdje se traži velika formalnost i dokazivost, ili gdje je promjena skupa i opasna. “Ipak, čak i tada se sve češće uvode iteracije i prototipovi, jer se pokazalo da je potpuno “zakucavanje” zahtjeva na početku rijetko realno” [1].

2.2. V-model i naglasak na verifikaciji i validaciji

V-model se često objašnjava kao “vodopad sa naglašenim testiranjem”. Na lijevoj strani V-a su aktivnosti specifikacije i dizajna, a na desnoj strani su odgovarajuće aktivnosti verifikacije i validacije. Ideja je da svaka faza razvoja ima svoj par provjere: zahtjevi se provjeravaju kroz prihvatno testiranje, dizajn sistema kroz sistemsko testiranje, dizajn modula kroz integraciono testiranje, a kod kroz komponentno ili jedinično testiranje.



Slika 6 - Dijagram V-modela životnog ciklusa razvoja softvera (SDLC)

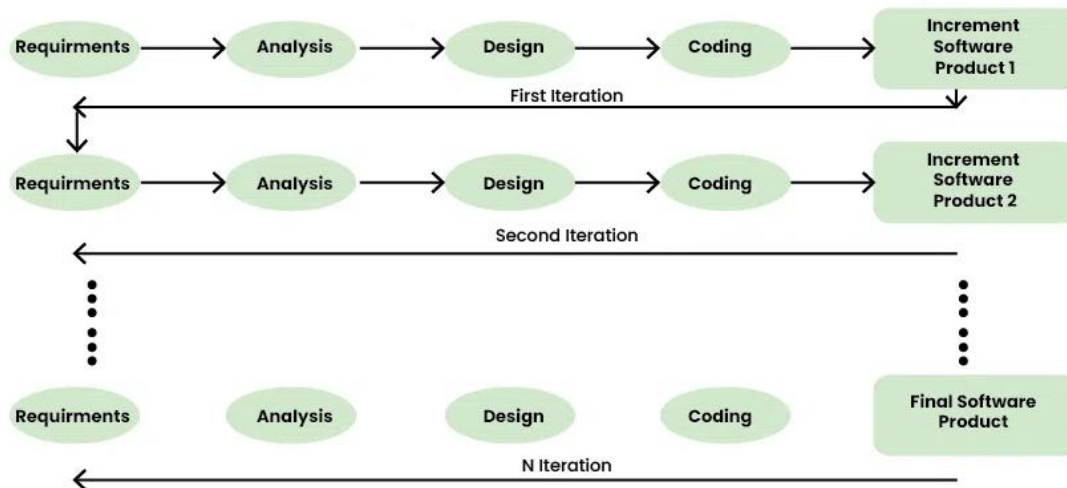
Poenta V-modela je da se o testiranju razmišlja ranije, a ne tek kad “sve bude gotovo”. To je dobra lekcija i za studenta, jer se često desi da se kod napiše bez razmišljanja kako će se testirati. Standard o verifikaciji i validaciji govori da se V&V procesi koriste da bi se utvrdilo da proizvod odgovara zahtjevima i da zadovoljava namjenu i potrebe korisnika, i naglašava da se V&V aktivnosti odnose i na sisteme koji se održavaju ili ponovo koriste. To je u skladu sa logikom V-modela: provjere nisu usputna stvar, nego planirani dio rada.

U praksi, V-model se često koristi u projektima gdje su zahtjevi formalniji i gdje se traži dokaz o provjerama, na primjer u industrijama sa većim rizikom. “Međutim, nije poenta da se V-model koristi samo “zbog papira”, nego da stvarno vodi do manje grešaka koje kasno isplivaju. Kasno otkrivene greške obično koštaju više, jer su već zahvatile više dijelova sistema” [2].

Ipak, i V-model ima ograničenja slična vodopadu, ako se tumači previše rigidno. Ako se zahtjevi stalno mijenjaju, a model je strogo linearan, tim može stalno “preslagati” dokumente i test planove. Zbog toga se u modernim timovima logika V-modela često primjenjuje unutar iterativnog ili agilnog okvira, gdje se u svakoj iteraciji radi i specifikacija i provjera, ali na manjem obimu.

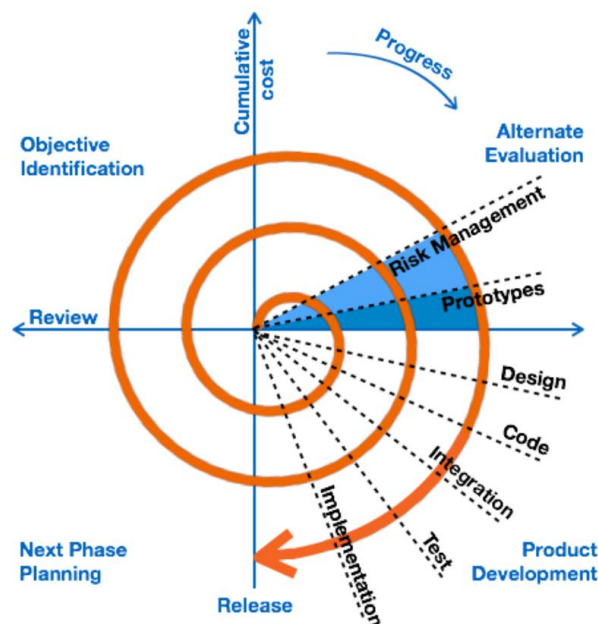
2.3. Iterativno-inkrementalni i spiralni pristup

Iterativno-inkrementalni razvoj znači da se sistem ne pravi odjednom, nego kroz više koraka. “Svaki korak donosi inkrement, odnosno dio funkcionalnosti koji je upotrebljiv. Iteracija znači da se kroz svaki ciklus ponovo prolazi kroz aktivnosti, ali na manjem komadu posla. Ovaj pristup se često pokazuje praktičnim jer korisnici ranije vide rezultat, a tim ranije dobije povratne informacije” [1].



Slika 7 - Dijagram iterativno-inkrementalnog modela razvoja softvera

Spiralni model je jedan poznat iterativni pristup koji naglašava upravljanje rizicima. U svakom krugu spirale tim identifikuje ciljeve, analizira rizike, razvije rješenje i procijeni rezultat prije nego što nastavi dalje.



Slika 8 - Dijagram spiralnog modela životnog ciklusa razvoja softvera (SDLC)

Iterativni pristup je često i most između klasičnih i agilnih metoda. Čak i timovi koji nisu “agilni” u formalnom smislu, često rade u iteracijama, jer je to realniji način da se nosi sa promjenama. “Standardi životnog ciklusa takođe dozvoljavaju prilagođavanje i izbor procesa zavisno od projekta, što pokazuje da postoji prostor za kombinovanje pristupa” [3].

Važna stvar kod iterativnog razvoja je disciplina oko upravljanja promjenama i kvaliteta. Ako se iteracije rade bez jasnog kriterijuma “šta znači gotovo”, može se desiti da se sistem stalno širi, a da kvalitet opada. Tu se opet vraćamo na proces: iterativnost pomaže fleksibilnosti, ali traži jasne provjere i kontrolu verzija.

3. AGILNI PROCESI I SAVREMENI NAČINI RADA

3.1. Agilne vrijednosti i principi

Agilni pristupi su nastali kao reakcija na previše “teške” procese u kojima se mnogo vremena trošilo na planove i dokumente, a premalo na stvarni proizvod. “Agilni manifest iz 2001. godine naglašava vrijednosti poput ljudi i interakcija, radnog softvera, saradnje sa korisnikom i spremnosti na promjenu” [4]. Ovo ne znači da plan i dokumentacija nisu važni, nego da nisu važniji od toga da softver radi i da korisnik dobije vrijednost.



Slika 9 - Dijagram 12 agilnih principa

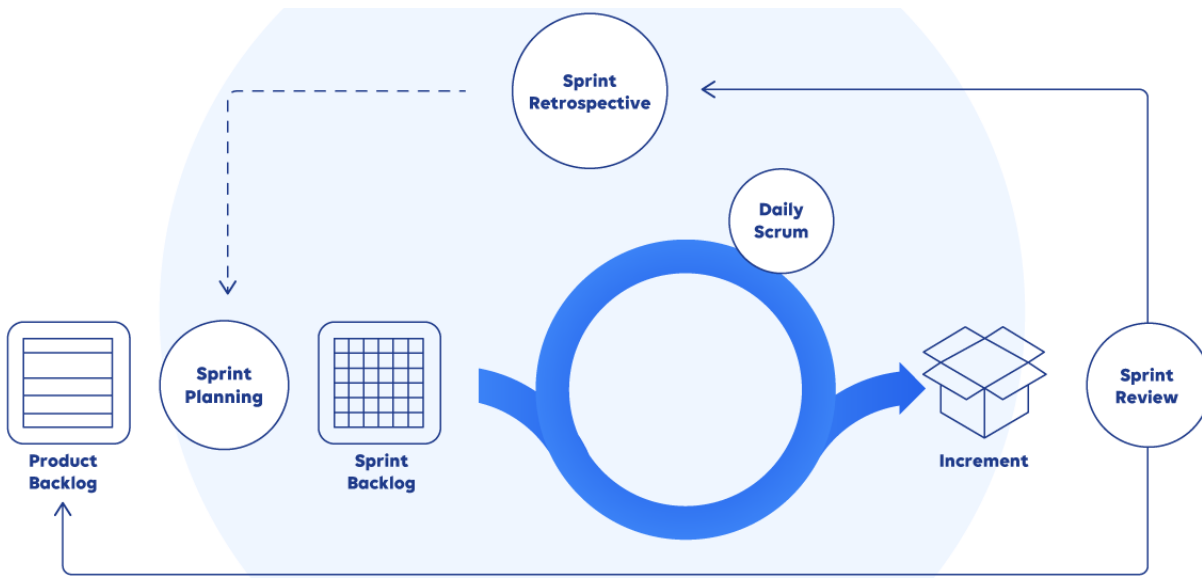
Praktično, agilnost znači da se radi u kraćim ciklusima, da se često dobija povratna informacija i da se plan prilagođava. Zahtjevi se ne tretiraju kao nešto što se “zaključa” prvog dana, nego kao nešto što se vremenom razjašnjava. Sommerville naglašava da se u savremenoj praksi često kombinuju planom vođene ideje i agilne metode, jer projekti rijetko spadaju u čist jedan pristup [1]. Agilni procesi takođe naglašavaju transparentnost i komunikaciju. U stvarnom radu to znači da se često razgovara, da se problemi iznose ranije i da tim pokušava da radi na onome što je najvrijednije. “Naravno, agilnost nije izgovor za nered. U ozbiljnim agilnim timovima postoji disciplina oko definicije gotovog, testiranja i upravljanja promjenama, samo je fokus drugačije raspoređen” [1].

3.2. Scrum kao okvir za razvoj proizvoda

“Scrum je jedan od najpoznatijih agilnih okvira. Scrum vodič ga opisuje kao okvir u kojem se radi u iteracijama zvanim sprintovi i naglašava elemente kao što su uloge, događaji i artefakti” [5]. Ono

što je korisno kod Scruma je što daje jednostavnu strukturu koja pomaže timu da se organizuje bez previše komplikovanja.

U praksi, Scrum se često koristi zato što omogućava da se rad podijeli na manje komade i da se napredak vidi često. Umjesto da se čeka šest mjeseci da se pokaže rezultat, tim pokušava da na kraju svakog sprinta ima nešto što se može demonstrirati. To ne mora uvijek biti kompletna funkcionalnost, ali treba biti provjerljiv napredak.



Slika 10 - Dijagram Scrum okvira za razvoj softvera

Scrum se u praksi skoro uvijek kombinuje sa tehničkim praksama kao verzionisanje, code review, automatizovani testovi, CI. Bez toga, sprint na kraju može biti samo “spisak gotovih tiketa”, ali ne i stabilan inkrement softvera.

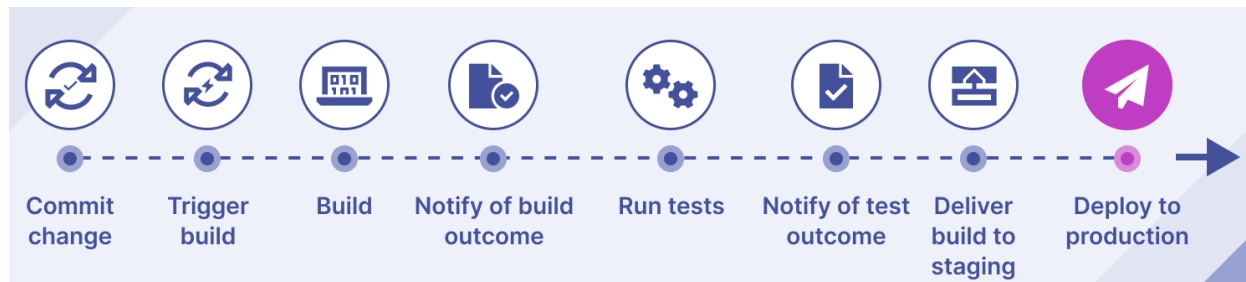
3.3. DevOps, CI/CD i uticaj na proces razvoja

DevOps se često objašnjava kao približavanje razvoja i operacija, sa ciljem da se softver isporučuje brže i sigurnije. U tom smislu, DevOps utiče na proces razvoja tako što “produžava” proces preko granice programiranja i testiranja, sve do isporuke i praćenja u produkciji. “Kontinuirana isporuka, kako je opisuju Humble i Farley, naglašava automatizaciju build-a, testiranja i raspoređivanja, kako bi se rizik smanjio i izdavanja postala češća i pouzdanija” [3].

U realnom svijetu, ovo mijenja način na koji tim planira i radi. Ako znamo da ćemo isporučivati često, onda su promjene prirodno manje. “Manje promjene se lakše testiraju i lakše se vraćaju ako nešto pođe po zlu. Takođe, to mijenja i odnos prema kvalitetu: testovi i provjere postaju dio svakodnevnog rada, jer pipeline neće pustiti izdanje koje ne prođe određene korake” [3].

“Istraživanja o performansama isporuke, popularizovana u knjizi “Accelerate”, naglašavaju da se uspješnost isporuke može posmatrati kroz brzinu i stabilnost, i da organizacije koje dobro isporučuju često imaju i bolje rezultate” [4]. Ne treba to shvatiti kao da će same metrike riješiti

probleme, ali je korisno kao ideja da proces razvoja treba mjeriti i poboljšavati, a ne samo “odraditi projekat” i zaboraviti.



Slika 11 - Dijagram toka CI/CD pipeline-a (kontinuirana integracija i isporuka)

U praksi, DevOps i CI/CD postavljaju i dodatne zahtjeve: konfiguraciono upravljanje mora biti uredno, okruženja moraju biti slična, a tim mora voditi računa i o bezbjednosti i o praćenju. “To znači da proces razvoja postaje “širi” i da traži malo više znanja i discipline, ali zauzvrat daje stabilnija izdanja i brže reakcije na probleme” [8].

4. UPRAVLJANJE PROCESOM I UNAPREĐENJE

4.1. Upravljanje zahtjevima i promjenama

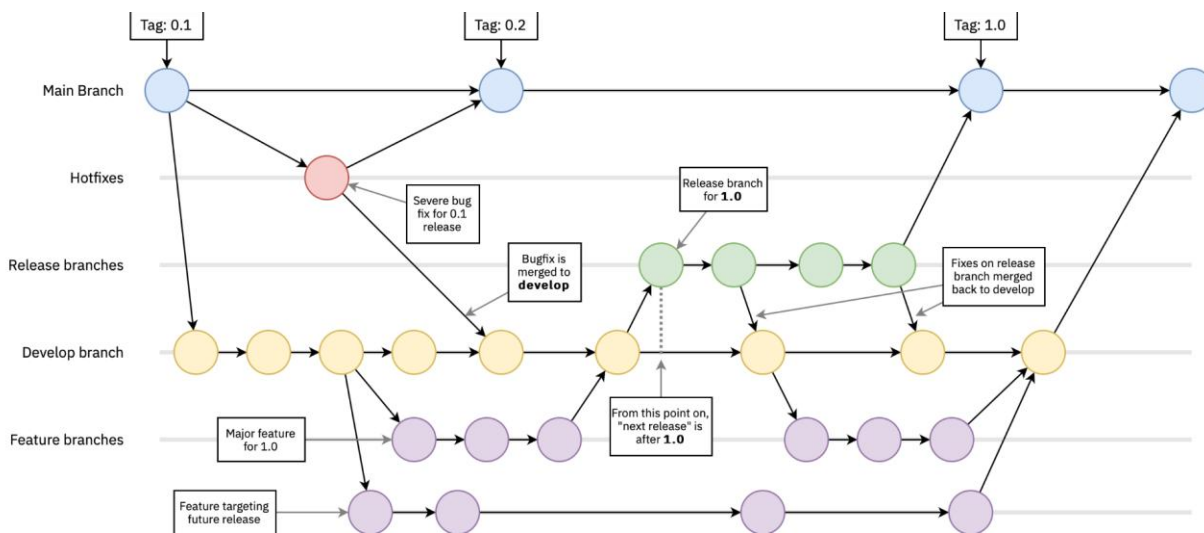
Upravljanje zahtjevima je dio procesa razvoja koji se bavi definisanjem, praćenjem i izmjenama zahtjeva tokom rada na softveru. Zahtjevi se rijetko ostaju isti od početka do kraja projekta, pa je potrebno imati način da se promjene kontrolišu.

Svaka promjena zahtjeva mora imati razlog, prioritet i odobrenje. Ako toga nema, dolazi do nereda u radu i do situacije gdje se stalno dodaju nove stvari bez jasnog plana. To direktno utiče na rokove i kvalitet sistema. Jedan čest problem je širenje obima projekta bez kontrole, poznato kao “scope creep”. To se dešava kada se u projekat stalno ubacuju nove funkcionalnosti, a vrijeme i resursi ostaju isti. Na kraju se dolazi u situaciju da dio posla ostane nedovršen ili urađen lošije.

Agilni pristupi pokušavaju da smanje ovaj problem tako što se rad dijeli na iteracije i što se prioriteti stalno ažuriraju. Ipak, i u takvom pristupu mora postojati kontrola promjena da bi se izbjeglo stalno mijenjanje plana. Dobar proces razvoja definiše kako se zahtjevi evidentiraju, kako se mijenjaju i ko ih odobrava. Na taj način se obezbjeđuje da promjene budu dio sistema, a ne ad hoc odluke.

4.2. Konfiguraciono upravljanje i praćenje rada

Konfiguraciono upravljanje se bavi praćenjem verzija softvera i svih njegovih dijelova. To uključuje kod, biblioteke, konfiguracije i izdanja sistema.



Slika 12 - Dijagram Git modela grananja (Git branching model)

Osnovni cilj konfiguracionog upravljanja je da se u svakom trenutku zna koja verzija sistema se koristi i koje su promjene uključene u tu verziju. Bez toga je teško pronaći uzrok greške kada sistem ne radi kako treba. U praksi se to rješava kroz verzionisanje koda, evidenciju izdanja i kontrolu promjena. Svaka izmjena treba da bude dokumentovana tako da se može pratiti ko je napravio promjenu i kada. Dokumentacija je dio procesa, ali ne treba da bude preobimna. Bitno je

dokumentovati ključne informacije kao što su arhitektura sistema, način pokretanja, važna pravila i ograničenja. Previše dokumentacije često se ne koristi, dok premalo dokumentacije otežava održavanje. Praćenje rada kroz tikete i backlog omogućava da se zna šta je urađeno i šta je preostalo. Tako se smanjuje rizik da se zadaci izgube ili ponavljaju.

4.3. Unapređenje procesa i zrelost

Proces razvoja softvera se može unapređivati tokom vremena. Cilj unapređenja je da se smanje greške, ubrza rad i poboljša kvalitet isporuke. Postoje modeli koji opisuju nivo zrelosti procesa, kao što je CMMI. Oni služe da se procijeni koliko je proces organizovan i gdje postoje slabosti koje treba poboljšati. Ideja nije dostizanje najvišeg nivoa, nego stalno poboljšanje u praksi.

Unapređenje procesa treba da se zasniva na stvarnim problemima. Ako se greške često otkrivaju kasno, uvode se bolji testovi i ranije provjere. Ako postoji problem sa verzijama, uvodi se strožije konfiguraciono upravljanje. Proces se ne unapređuje zbog formalnosti, nego zbog konkretnih potreba projekta. Ako se uvede previše pravila bez potrebe, proces postaje spor i težak za korištenje.



Slika 13 - Dijagram nivoa zrelosti CMMI modela

SWEBOK i slični vodiči služe kao pregled oblasti koje postoje u softverskom inženjerstvu. Oni mogu pomoći timu da prepozna koje dijelove procesa treba poboljšati i kako da se organizuje rad na bolji način.

ZAKLJUČAK

Proces razvoja softvera predstavlja organizovan način rada kojim se od ideje dolazi do gotovog i funkcionalnog softverskog sistema. On povezuje ljude, aktivnosti i pravila u jednu cjelinu. Osnovne aktivnosti u svakom procesu su prikupljanje zahtjeva, dizajn, implementacija, testiranje i isporuka.

Ne postoji jedan univerzalan model koji odgovara svim projektima. Različiti modeli procesa samo na različit način organizuju iste aktivnosti. Klasični modeli daju strožiju strukturu, dok iterativni i agilni modeli uvode veću fleksibilnost i lakše prilagođavanje promjenama. Modeli kao što su vodopad i V-model pogodni su za situacije gdje su zahtjevi stabilni i jasno definisani. Spiralni i iterativni modeli omogućavaju bolje upravljanje rizicima kroz ponavljanje i postupno unapređenje. Agilni pristupi stavljaju naglasak na brzu povratnu informaciju i stalnu saradnju sa korisnikom.

Scrum predstavlja jedan od najčešće korištenih agilnih okvira, gdje se rad organizuje u kratke iteracije i gdje se prioriteti redovno ažuriraju. DevOps i CI/CD proširuju ovaj pristup tako što povezuju razvoj i isporuku kroz automatizovane procese.

Osnovna poruka je da proces razvoja softvera treba da pomogne radu, a ne da ga otežava. Dobar proces smanjuje greške, olakšava upravljanje promjenama i čini isporuku predvidivom. Loše definisan proces može usporiti razvoj i stvoriti dodatne probleme. Zbog toga se proces uvijek bira i prilagođava prema projektu, veličini tima i vrsti sistema. Najvažnije je da se stalno poboljšava na osnovu iskustva i stvarnih problema koji se javljaju tokom razvoja.

POPIS SLIKA

Slika 1 - Dijagram faza životnog ciklusa razvoja softvera (SDLC)	2
Slika 2 - Jednostavan dijagram faza razvoja softvera	3
Slika 3 - Dijagram ISO 25010 modela kvaliteta softvera	4
Slika 4 - Dijagram PDCA ciklusa za kvalitet softvera (Plan–Do–Check–Act).....	4
Slika 5 - Dijagram vodopadnog modela (Waterfall model).....	5
Slika 6 - Dijagram V-modela životnog ciklusa razvoja softvera (SDLC)	6
Slika 7 - Dijagram iterativno-inkrementalnog modela razvoja softvera.....	7
Slika 8 - Dijagram spiralnog modela životnog ciklusa razvoja softvera (SDLC)	7
Slika 9 - Dijagram 12 agilnih principa.....	9
Slika 10 - Dijagram Scrum okvira za razvoj softvera	10
Slika 11 - Dijagram toka CI/CD pipeline-a (kontinuirana integracija i isporuka).....	11
Slika 12 - Dijagram Git modela grananja (Git branching model)	12
Slika 13 - Dijagram nivoa zrelosti CMMI modela	13

CITATNI IZVORI

- [1] I. Sommerville, Software Engineering, Boston, MA: Pearson, 2015.
- [2] R. S. Pressman i B. R. Maxim, Software Engineering: A Practitioner's Approach, New York, NY: McGraw-Hill Education, 2019.
- [3] ISO/IEC, »ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models,« International Organization for Standardization (ISO), Geneva, 2011.
- [4] Agile Manifesto Authors, »Manifesto for Agile Software Development,« Agile Manifesto, 2001. [Mrežno]. Available: <https://agilemanifesto.org/>.
- [5] K. Schwaber i J. Sutherland, »The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game,« The Scrum Guide, 2020. [Mrežno]. Available: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>.
- [6] J. Humble i D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Upper Saddle River, NJ: Addison-Wesley Professional, 2010.
- [7] N. Forsgren, J. Humble i G. Kim, Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, Portland, OR: IT Revolution Press, 2018.
- [8] CMMI Institute, »The CMMI Institute Announces CMMI Development V2.0,« CMMI Institute, 2018. [Mrežno]. Available: <https://cmmiinstitute.com/news/press-releases/march-2018/announcingv2>.